

# Huey-Core (iMac 5K 2017) Node – Technical Overview

## System Overview

Huey-Core is a **2017 iMac 27" 5K** repurposed as a core node for the Monkey-Head Project. The hardware provides a 27-inch 5K Retina display and robust internals (quad-core Intel CPU with up to 8 threads, discrete AMD Radeon Pro GPU, and upgraded **48 GB DDR4 RAM**) <sup>1</sup>. It features a hybrid storage setup (Apple Fusion Drive architecture with a fast NVMe SSD and a high-capacity HDD) which is leveraged for both performance and persistence.

In the project's cluster, Huey-Core serves as the **primary control and interface node**, essentially the "brain" and console for the HueyOS system. It runs **Debian 13 "Trixie"** Linux (planned upgrade to Debian 14 "Forky") with a full desktop environment <sup>1</sup>. This node acts as a universal display and input station for the robotic AI/OS – operators can directly interact with HueyOS's UI on the iMac, and it also orchestrates or monitors other nodes. Networking is typically via gigabit Ethernet (bonding or Wi-Fi fallback as needed), and remote access (for headless operation) is facilitated through VNC/SSH if required, although Huey-Core usually provides a local GUI console <sup>2</sup>.

The desktop environment is **GNOME on Xorg** <sup>1</sup>. Xorg was chosen over Wayland to ensure compatibility with tools like TigerVNC and certain GPU-accelerated applications. On boot, the system automatically logs in the service account (e.g. `d1rp`) and launches the HueyOS interface – a retro-themed, green-on-black console window for the AI's "consciousness" output alongside verbose logs (part of Huey's design ethos). This iMac effectively becomes the face of the robot, blending modern Linux capabilities with a nostalgic terminal aesthetic.

## Custom Linux Kernel 6.17.5 (Huey Build)

To fully support the Mac hardware and project requirements, Huey-Core runs a **custom Linux 6.17.5-huey+ kernel**. This kernel is based on upstream 6.17.x but built with a specialized configuration and patches tailored for HueyOS. The build is **low-latency optimized** and includes targeted drivers for Apple-specific components and AMD GPUs <sup>2</sup>. For example, support for the Cirrus Logic audio codec (CS8409) used in iMacs is enabled in-kernel for HDA (Intel High Definition Audio) operation <sup>3</sup> <sup>4</sup>. The Huey kernel configuration makes the `snd_hda_codec_cs8409` driver a loadable module, along with related codecs like `snd_hda_codec_cirrus`, ensuring the audio hardware is recognized. These drivers were not fully available or stable in earlier stock kernels, so including them was critical for out-of-the-box sound on the iMac <sup>5</sup>.

Other kernel customizations include keeping the newer Sound Open Firmware (SOF) audio stack available as a fallback but **preferring the legacy HDA path** for the CS8409. In practice, this means the Huey kernel build blacklists certain SOF modules to avoid conflicts. A modprobe configuration (e.g. `/etc/modprobe.d/`

90-hda-preferred.conf) is deployed to blacklist `snd_sof` drivers (such as `snd_sof_pci`, `snd_sof_intel skl`) so that the in-kernel HDA driver claims the device <sup>6</sup>. Additionally, that config disables any digital microphone detection (`dmic_detect=0`) – since iMacs don't use Intel DMIC – and sets `power_save=0` on `snd_hda_intel` to prevent audio power-management pop noises on the speakers <sup>7</sup>. These tweaks ensure a clean initialization of the audio codec without the quirks that SOF or power-saving would introduce.

The kernel is compiled with the **PREEMPT\_DYNAMIC low-latency settings** (to improve responsiveness for real-time audio and interactive tasks) and with unneeded components trimmed out. Notably, the AMD GPU driver (AMDGPU) is included for the Radeon Pro graphics, and support for Broadcom Wi-Fi/Bluetooth is present (firmware blobs are loaded in user space as needed). Building the kernel was done via the usual make process (using `localversion=-huey` to label it) and packaged for Debian. The end result is a **DKMS-free** kernel package – all necessary drivers are built in or as modules in-tree, avoiding external module builds <sup>3</sup>. After installation, GRUB was updated to add the new kernel, and an initramfs was generated for it <sup>8</sup>.

Each Huey node uses a similar 6.17.x-huey kernel build adjusted for its hardware. For Huey-Core's iMac (model identifier iMac18,3), this custom kernel was particularly important to address the audio and GPU support. During bring-up, the kernel command line was also adjusted: the **splash screen was disabled** and verbose boot enabled so that one can see boot logs on the 5K display (useful for troubleshooting and also aligning with the retro-computing vibe) <sup>9</sup>. The resolution is set conservatively to 2560×1440 during boot to avoid mode-switch issues on the high-DPI screen before the display server starts. In summary, the 6.17.5-huey kernel provides Huey-Core with a solid foundation: support for all hardware devices, low-latency operation, and no reliance on out-of-tree modules or DKMS hacks.

## Audio System Bring-up (CS8409 & PipeWire)

Audio on the 2017 iMac presented one of the more complex bring-up challenges due to the **Cirrus Logic CS8409 audio codec** and its companion amplifiers. Out of the box, a standard Linux install would show “Dummy Output” with no sound, as the CS8409 requires specific codec initialization sequences not originally in older kernels. With the custom kernel in place (which includes the necessary codec driver), the device is recognized as an HDA codec. The **HueyOS audio architecture** then builds on top of that using PipeWire for audio routing (replacing the older PulseAudio).

The **HDA codec path** is used exclusively on Huey-Core, bypassing Sound Open Firmware. This decision was made because the in-kernel support for CS8409 via HDA had proven more reliable after some tuning, whereas SOF support for Apple T2/CS8409 was either experimental or intended for newer Macs. By blacklisting SOF as mentioned, we ensure the `snd_hda_intel` driver binds to the iMac's audio device <sup>6</sup> and applies any Apple-specific quirks that have been added to the HDA driver. Some of these quirks handle things like channel mapping and basic pin configuration for the iMac's speakers and headphone jack, but additional tweaking was still needed for full audio functionality.

After the first boot with the Huey kernel, the CS8409 codec was detected (visible in `dmesg` logs with Apple-specific messages) and two sound cards appear – one for the AMD HDMI audio (unused) and one for the Cirrus/Intel PCH HDA. To get sound from the iMac's internal speakers, Huey-Core runs an **audio setup**

**script at user login** (the “PipeWire default sink script” noted in project docs) <sup>10</sup>. This script performs a series of actions:

- **Restart PipeWire/WirePlumber services** – ensuring the audio graph is fresh. For example, it runs `systemctl --user restart wireplumber.service pipewire.service pipewire-pulse.service` to reset the PipeWire daemon and session manager.
- **Set the default audio sink** to the Cirrus analog output. PipeWire sometimes enumerates the HDMI output first, so the script calls `pactl list short sinks` and then `pactl set-default-sink 0` (assuming index 0 is the internal sound card) <sup>11</sup>. This directs all audio output to the iMac’s speakers.
- **Unmute and set volume** on key ALSA mixer controls. Using `amixer`, the script bumps Master, Speaker, and Headphone volumes to ~80% and ensures they are unmuted <sup>12</sup>. It also disables “Auto-Mute Mode” (which on PCs would mute speakers when headphones are plugged, but on this iMac it’s safer to handle manually) <sup>13</sup>.
- If after these steps sound is still not heard, the script can invoke `hda-verb` **tweaks** to nudge the codec. On Macs, it’s known that writing certain coefficients can enable the speaker amp. For instance, sending verbs to node 0x47 (the codec’s processing widget) to set a certain coefficient index and value is a known fix in community scripts. In Huey-Core’s case, the need for `hda-verb` was largely mitigated by the updated kernel quirks, but the infrastructure is there. The audio setup script will try a sequence of `hda-verb` commands if sound is not active – effectively poking the codec to power up the internal speaker amplifiers or to switch output paths. (These are low-level hardware commands and were gathered from Apple codec reverse-engineering efforts in Linux forums.)

Through this combination of kernel support and user-space initialization, **audio playback on Huey-Core is achieved**. The internal stereo speakers work and the 3.5mm headphone jack is functional, though jack detection might not automatically switch; if a physical disconnect isn’t auto-detected, the script can also explicitly toggle outputs via `amixer` or `hda-verb`. The **microphone input** (in iMac’s case, the built-in mic is on the webcam and a digital input) is currently not a focus – with `dmic_detect=0` the digital mic might be disabled, so Huey-Core uses external USB audio or the Mac’s line-in if needed for audio capture.

It’s worth noting this solution, while effective, is a **temporary workaround**. The project considers the CS8409 support an open issue: upstream Linux is gradually improving support for these Cirrus codecs. HueyOS maintainers have “mitigation scripts” in place and track a long-term fix in the kernel roadmap <sup>14</sup>. Future kernel updates (e.g., 6.18 or newer) may bring official fixes that make manual verbs unnecessary <sup>15</sup> <sup>16</sup>. For now, Huey-Core’s audio is stable for everyday use (no dummy output, good volume levels), after this one-time setup each boot or login. PipeWire manages audio streams (e.g., text-to-speech output from Huey’s AI or alert sounds) and would also allow routing to Bluetooth speakers or other outputs if needed.

## “RAM Mode” Integration with SquashFS Overlays

One distinctive feature of Huey-Core’s software setup is the ability to run the OS in **RAM Mode**, where the entire operating system loads into RAM at boot and runs without constant disk access. This design is inspired by live-boot environments and provides both performance benefits and resilience. The implementation uses a **SquashFS** compressed image of the root filesystem combined with an **overlay filesystem** for write operations.

**Partitioning for RAM Mode:** During installation, the Huey-Core node was set up with a special partition layout (dubbed the “huey-key” layout) to facilitate this. In summary, there is a relatively small partition (around 16 GB) that holds the core OS image, and a large partition (100+ GB) allocated for persistent data <sup>17</sup>. The 16 GB partition contains a SquashFS file (a read-only compressed snapshot of the OS) and the necessary boot files. The 100 GB partition is formatted as a writable filesystem (ext4 or Btrfs) and labeled for persistence. Additionally, a 1 GB EFI System Partition holds GRUB, and a 4 GB swap partition exists for completeness <sup>17</sup>. This scheme ensures that the OS can be treated as an immutable image, while all changes (installing packages, config tweaks, user data) reside on the persistence volume.

**Initramfs and Boot Process:** Huey-Core’s GRUB menu includes two boot options for the HueyOS installation – a standard mode and a “Live RAM Mode”. The **RAM Mode** entry passes special kernel parameters to activate the live-boot mechanism. For example, it uses `boot=live` along with `persistence` and `toram=1` flags on the kernel command line <sup>18</sup>. When selected, the GRUB configuration directs the system to find the SquashFS image and load it into memory. Debian’s **live-boot** scripts (included in the `initrd`) detect these parameters and handle the setup: - The `toram` option triggers the `initramfs` to copy the entire SquashFS image into RAM early in the boot sequence. This can take a short while (the image might be several gigabytes, compressed – loading from SSD is fast, from HDD a bit slower, but 48 GB of RAM easily accommodates it). - The `persistence` option tells live-boot to look for a persistence partition (usually identified by a label “persistence” or a file `persistence.conf`). It then mounts this partition and sets up an **OverlayFS**: the SquashFS image is the lower (read-only) layer and the persistence partition becomes the upper (read-write) layer. This way, as the system runs, any file changes are written to the persistence disk, while reads for unchanged files come from the in-RAM SquashFS. In effect, the running root filesystem is a union of the base image and the persistent changes.

After this `initramfs` setup, the system hands off to Debian as usual but now fully operational in RAM. In this mode, the OS can run with minimal disk access – the internal drives are mostly idle since reads are served from memory and writes go to the overlay (which could be the HDD, unless an in-memory `tmpfs` is used for write layer – however, by default `persistence` implies using the disk for writes to actually persist them). One benefit here is speed: read-intensive operations (loading libraries, launching programs) come straight from RAM (which is orders of magnitude faster than HDD). Another benefit is that it isolates the base system from corruption – if something goes wrong, a reboot will load the known-good SquashFS again.

**Standard Mode vs RAM Mode:** In the standard boot mode, Huey-Core can either boot directly off the disk (using the 16 GB partition as a normal root filesystem) or it can still utilize the SquashFS but without copying to RAM (i.e., run the SquashFS in-place from disk with the same overlay on persistence). The exact configuration chosen for standard mode in HueyOS is to **run as a persistent live system even on the internal drive**, for consistency. This means even the normal boot might be using `boot=live` (but without `toram`), effectively reading from the compressed image on disk. This approach was hinted by the project’s “huey-key” concept, treating even internal installs as live images. The advantage is that the environment is identical whether booted normally or in RAM mode; the only difference is whether the SquashFS stays on disk or is loaded into memory.

From a user perspective, GRUB likely lists something like: - “HueyOS Desktop (Standard)” – which boots the HueyOS with persistence, running from disk. - “HueyOS RAM Mode” – which boots with `toram` to load the OS into RAM.

Additionally, there's mention of an **EFI Live USB** fallback <sup>19</sup>. The team prepared a bootable USB drive (the "Huey key") that contains a live HueyOS environment with the same partition structure and an ISO9660 image. GRUB on Huey-Core can detect and chainload this if needed (for recovery or cloning the system to new hardware). The live USB environment uses the same mechanism (SquashFS + overlay), and having it integrated in GRUB means an operator could reboot the iMac into the external live system for maintenance or upgrades (for example, when doing the Debian 14 upgrade, they might boot the live key to modify the internal disks safely).

**Initramfs Hooks:** Under the hood, enabling this functionality required installing Debian's **live-boot** and **live-config** packages and configuring persistence. A custom `persistence.conf` likely specifies which directories are persistent (common choices are to persist everything or specific paths like `/home`, `/var/log`, etc.). Huey-Core appears to persist most changes (since the goal is a usable desktop, not a throwaway live session). The initramfs was updated to include the **overlay and squashfs modules** and the live-boot scripts. These live-boot scripts recognize the `findiso=` or device UUID provided via GRUB to locate the SquashFS image on the root partition <sup>20</sup>. There might also be custom hooks – for example, to automate something like copying the image to RAM conditionally. But given Debian's robust live framework, likely little custom code was needed beyond configuration.

In operation, **RAM Mode** has been successfully tested on Huey-Core. The system can run completely diskless once booted – an interesting scenario for a desktop iMac. It ensures that even if the internal HDD is slow or spins down, the user experience remains snappy. It also means Huey-Core could, in principle, run off a read-only medium or be quickly reset by rebooting (since non-persistent changes can be thrown away if desired by not saving to the persistence partition or by booting without it). This is in line with the project's emphasis on resilience and reproducibility: governance remains decentralized while memory remains unified, and here "memory" is literal – the unified state can live in RAM for speed.

## Storage and Partition Layout

Huey-Core's internal storage is set up to balance performance (SSD) and capacity (HDD), using a partitioning scheme that isolates different aspects of the system. As mentioned, the **SSD contains the boot and OS image**, while the **HDD contains most of the writable data**. Specifically, the partition layout is as follows

<sup>17</sup> :

- **EFI System Partition (ESP)** – 1 GiB FAT32 on the SSD, mounted at `/boot/efi`. This holds the GRUB bootloader and UEFI firmware files. The large size (1 GiB) is generous, leaving room for multiple kernel images, perhaps backups or Windows boot files if needed in the future.
- **Swap Partition** – 4 GiB on SSD. Swap is infrequently used given 48 GB RAM, and it's on SSD for faster access when needed. The 4 GiB size is not enough for hibernation (which would require ~48 GB), but hibernation isn't a use-case here; the swap is mainly for kernel memory overflow or suspend-to-RAM support.
- **Root/System Partition** – ~16 GiB on SSD, formatted with either **Btrfs** or **ext4**. This partition holds the core OS. In the Huey live configuration, this may not be a traditional root filesystem; instead it likely contains a `filesystem.squashfs` file (the compressed image of the OS) along with the `/boot` directory (vmlinuz and initrd). If Btrfs is used, it could also store subvolumes for different versions of the OS image (snapshotting an old image before replacing it, for example). However, ext4 is perfectly plausible here since the usage is mostly read-only. 16 GiB has proven sufficient for a

Debian system with GUI and some extras – currently HueyOS's base image is well under that size, and any larger data (like AI model files) are stored on the persistence partition.

- **Persistence/Data Partition** – 100 GiB on the HDD, likely ext4. This partition is labeled `persistence` and mounted as the upper layer for the overlay. It contains all the variable data: when the system is running, any file modifications actually land here. It means effectively this partition will have the `/etc`, `/var`, `/home` changes, plus any additional software installed post-image. Because it's large, the team can install development tools, store logs, and even datasets without worrying about space (the iMac's HDD might actually be 1 TB or more, so there may be unallocated space or the partition could be expanded in future). The documentation referred to this as "persistence" which aligns with live-boot terminology <sup>21</sup>.
- **(Optional) Encrypted Secrets Partition** – possibly a small LUKS-encrypted partition either on SSD or HDD dedicated to sensitive material (credentials, encryption keys for the governance model, etc.). The partition scheme mentions this as optional <sup>17</sup>. If present, it would be manually mounted when needed (e.g., mounted at `/huey/secrets` after entering a passphrase). This provides an extra layer of security for critical data in case the device is lost or compromised.

With this setup, **fstab** in the live context is minimal – the live-boot process mounts the persistence partition automatically. If Huey-Core were converted to a normal installed system mode, the `fstab` would explicitly mount the root (for Btrfs, something like `UUID=<...> / btrfs subvol=@,defaults 0 1` and similarly for subvolumes). But since Huey-Core runs as a live system, the persistence mechanism supersedes the need for many static `fstab` entries. The likely entries in `fstab` are only for the EFI partition (mounted at `/boot/efi` for kernel updates) and swap. The persistence partition might not even appear in `fstab` if managed by label – or it could be referenced by label in a persistence config file.

It's also worth noting that Apple's Fusion Drive typically would present as a CoreStorage volume or, to Linux, as separate drives (an SSD and an HDD). In Linux, they show up as separate block devices. Here, the project chose not to attempt a RAID or LVM spanning both; instead, they used them discretely to each do what they do best. This avoids the complexity of bcache or LVM caching – simpler and effective.

Additionally, **Btrfs subvolumes** were mentioned in passing in project notes, which suggests Btrfs might be in use on at least one of the drives. If, for instance, the HDD persistence partition is Btrfs, they could create subvolumes for different data sets or snapshots (e.g., a subvol for `/huey/memory` data vs. general home files). No explicit list of subvolumes is given, but the door is open to use Btrfs's features for snapshotting the overlay state or for organizing data. The HueyOS team values reproducibility, so one can imagine a future feature where, before a major upgrade, they snapshot the current state, allowing rollback if needed (this would require Btrfs for the persistence layer).

In summary, Huey-Core's partition layout segregates the system software from user and mutable data. This not only aligns with the live OS approach but also makes maintenance easier – for example, the 16 GB system image can be backed up or replaced in one go, and the large data partition can be separately managed or reused if the OS is re-imaged. It also means the SSD is used mostly for read-mostly operations (extending its lifespan) while the HDD handles the write-heavy workload (with plenty of space to spare).

## Software Stack and Services

On top of the base OS, Huey-Core runs the Monkey-Head Project's **HueyOS software stack**, which is a combination of custom Python services, AI models, and standard Linux userland utilities. The goal is to provide a smooth integration between the system and the Huey cognitive layers.

**Desktop Environment & GUI:** The GNOME desktop is primarily a host for the Huey UI. The user auto-login launches a terminal (or a pair of terminals) with Huey's interface. One panel might show the high-level "Sovereign" output (the AI's primary messages), and another might tail log files or diagnostics. This UI setup is custom but built using common tools (possibly tmux splits, or a small Tkinter/PyQt app). There is also a control GUI accessible via web (the FastAPI endpoints for HueyOS) – the iMac can be used to open a browser (Microsoft Edge Beta is the standardized choice) to an ops console. Edge Beta is installed on Huey-Core as the default browser across Linux nodes <sup>22</sup>.

**HueyOS Services:** The core runtime (`huey run`) is a Python orchestrator that starts multiple agent processes (Spark, Zap, etc., the AI agents) along with background services. On Huey-Core, these likely run in user space (rather than system services) for ease of development. For instance: - The **Huey API** (a FastAPI server) runs via Uvicorn, binding to localhost (port 8000) <sup>23</sup>. This could be launched either as a user systemd service or via a `uvicorn` command in a terminal. It provides endpoints for status, task submission, sensor data, etc., which can be accessed from anywhere in the network (with SSH tunneling or proper firewall rules). - The **Scheduler/Task Manager** is part of the core and can dispatch tasks to different "agents" (potentially even across nodes). Huey-Core, being central, might coordinate tasks to other nodes (like Huey-Portable or Huey-Prime) or handle them itself if they're assigned to it. The API includes routes for task management (submit, cancel, query) <sup>24</sup> <sup>25</sup>. - **Memory Hive:** HueyOS uses a unified memory approach, logging JSON records and storing context in SQLite databases. On Huey-Core, these files are located under `/huey/memory/` (which likely resides on the large persistence partition, possibly as a bind mount or simply as part of the overlay) <sup>26</sup>. A background service or timer may handle memory log rotation or syncing to backups. For example, a systemd timer could periodically run `huey memory-sort` to organize or compress logs (the CLI has such commands <sup>27</sup>). - **Sensor & Hardware Services:** Although an iMac doesn't have a plethora of external sensors, Huey-Core might still run a "sensor plugin" service that fakes or aggregates data. In the Huey architecture, there is support for sensor plugins (e.g., a dummy temperature sensor plugin) <sup>28</sup>. On actual robot hardware, sensors would interface via Arduinos or other microcontrollers, but Huey-Core can simulate or simply log the incoming data. If any physical sensors are connected (say via USB or network), those would be handled by daemon processes as well. The project's code structure (e.g., `huey.hardware.plugins`) suggests extensibility for sensors <sup>29</sup>. - **User Services and Autostart:** Aside from the audio setup script, there are other custom startup scripts. One example is the **VNC service**. The documentation describes a TigerVNC setup where Huey-Core (earlier called huey-portal) can act as a VNC client to connect to a headless node (huey-legacy) <sup>30</sup>, but also the iMac could run a VNC server for remote access. A systemd service template `tigervnc@.service` was prepared for this purpose <sup>31</sup>. Enabling it on Huey-Core means one could remote into the GNOME session (it likely launches on demand or on boot in multi-user target). The VNC is configured to only accept local connections (`- SecurityTypes None`), relying on SSH tunnels for security <sup>32</sup>. This way, developers can SSH into Huey-Core and forward VNC to see the desktop if they can't be at the console. - Another service tweak was masking `iiio-sensor-proxy.service` on devices where it's irrelevant <sup>33</sup>. On Huey-Core, the iMac has an ambient light sensor, but it's not crucial; and on Huey-Portable, the screen rotation sensor could cause annoyances, so it's turned off by default. These little adjustments help reduce CPU wakeups and noise from unused services.

**Audio Routing:** We discussed output routing; additionally, if Huey-Core needs to perform Text-To-Speech (TTS) or other audio output (alerts), PipeWire handles it and the default sink is the internal speakers. If voice recognition or other microphones are used, an external USB mic might be employed until internal mic support is confirmed. BlueZ is available for Bluetooth audio if needed (e.g., connecting to a speaker or headset), though not a primary use-case.

**Security and Maintenance:** By default, SSH is enabled (key-based auth, with a lab-wide default key configured for convenience) <sup>34</sup>. Password login for root is disabled. Sudo is set up for the main user. Firewalls are likely managed by the router, but Huey-Core itself isn't exposed directly to the internet (offline-first philosophy). There's a mention of rotating the default lab SSH key and ensuring personal keys are added <sup>35</sup> – an operational task to maintain good security hygiene.

Regular Linux user applications (file managers, text editors, etc.) are installed as needed but the system was “debloated” to remove unnecessary packages (e.g., LibreOffice, games, printing services) to save space and resources <sup>36</sup>. Only essential packages plus project-specific ones (Python libs for AI, dev tools for kernel building) are present. The list of core packages for Huey-Core includes compilers and kernel build tools, as the team can compile kernels or other software directly on this machine if needed <sup>37</sup>.

In effect, Huey-Core runs a fairly lean yet fully functional Linux desktop, with Monkey-Head Project's HueyOS layered on top. The combination of systemd for services, GNOME for UI, and Python for the AI logic makes it easy for the developers to manage and extend. As development continues, some of these services may be containerized or moved into Kubernetes (there are k8s manifests in the repo, hinting at future clustering) <sup>38</sup>, but for now on this single node, running them directly is simpler.

## Bootloader Configuration (Dual-Mode Boot)

The GRUB UEFI bootloader on Huey-Core is configured to support the dual-mode boot described earlier. When the system powers on, GRUB presents a menu with at least two HueyOS entries and possibly some tools. The primary entry (default) boots HueyOS in the standard desktop mode (disk-based or normal persistence), and the second entry boots HueyOS in RAM Mode.

Under the hood, these entries are defined either in `/etc/grub.d/40_custom` or a custom `grub.cfg`. For example, a **custom GRUB menuentry** for RAM Mode might look like:

```
menuentry "HueyOS (Live RAM Mode)" {
    insmod linux
    insmod gzio
    search --no-floppy --fs-uuid --set=root <UUID-of-OS-partition>
    echo    "Loading HueyOS (SquashFS) into RAM..."
    linux   /live/vmlinuz-x86_64 boot=live persistence toram=1 quiet splash=0
    initrd  /live/initrd.img
}
```

This is analogous to the live USB entry found in project files <sup>18</sup>, adjusted for the internal drive. The `search` command finds the partition by UUID (since device names can change). The kernel



(`vmlinuz-6.17.5-huey+`) and `initrd` are loaded from the `/live/` directory or directly from `/boot/` depending how the system was set up. Key parameters include `boot=live`, `persistence`, and `toram`. In contrast, the **standard entry** might omit `toram` (and possibly even `boot=live` if using a normal install). However, given the implementation, it's likely the standard entry still uses `boot=live` `persistence` but no `toram`, causing it to mount the SquashFS from disk. Both entries use the same kernel and `initrd` – the difference is just the flags.

GRUB is set to a short timeout (5 seconds) and will autoboot into standard mode if no selection is made <sup>39</sup>. The user can interrupt and choose RAM Mode when desired (for instance, for a demo or if they want to reduce wear on the HDD during a long session). Additionally, GRUB includes an **Advanced Options** submenu for troubleshooting. This typically would have entries like recovery mode or older kernels (if any) <sup>40</sup>. Since Huey-Core is single-booting Debian, no Windows or macOS entries are present – the iMac's macOS was likely replaced entirely by this custom setup.

One more entry the team planned is the **"EFI Live USB"** option <sup>19</sup>. This would be a menu entry that appears only if a specific USB stick (the Huey key) is plugged in. It could be implemented via a custom script that detects the USB by label/UUID in GRUB and then shows a menuentry to boot from it (using `configfile` or chainloading). Alternatively, they might rely on UEFI boot menu for that. The Action Plan item suggests integrating it into GRUB menu for convenience, so an operator at the iMac doesn't have to fiddle with UEFI boot order – they can select the USB fallback from the GRUB screen if needed.

Since Huey-Core's kernel is custom, GRUB's config ensures the kernel command line has all needed parameters consistently. For instance, it passes `quiet` only if desired (they have an item to ensure `quiet splash` is removed to show boot logs <sup>41</sup>, so likely the entries do **not** include `quiet` or `splash`). Instead, one sees a verbose boot (which suits a development machine). Another parameter that might be present is `mem_sleep_default=deep` or similar power settings, but those are minor.

After GRUB hands off to the kernel, if the **toram** mode was selected, it prints a message about copying to RAM (the custom echo in the entry or by live-boot script). This feedback is useful so the user knows what's happening, especially if it takes, say, 20 seconds to copy the image – otherwise it might appear to hang.

In summary, the GRUB configuration on Huey-Core is carefully tailored for dual boot modes: - **Standard Desktop Mode**: boots normally into the Debian system (with HueyOS) using the persistence partition for all writes. This is the everyday mode and is relatively fast to boot. - **RAM Mode**: boots into a variant where the OS runs entirely from RAM, ideal for certain scenarios (perhaps when Huey-Core is moved and no reliable disk access, or to eliminate disk IO for performance). This mode uses the same root image but preloads it into memory. - **Recovery/Tools**: additional options like a failsafe (non-persistent) mode can be available. The provided grub template on the live media includes a *fail-safe mode* without persistence (and with minimal drivers) <sup>42</sup>, which could be mirrored on internal setup for troubleshooting if the overlay gets messed up. Also a memtest entry might exist (GRUB auto-detects Memtest86+ if installed) <sup>43</sup>.

GRUB itself is installed in UEFI mode on the iMac (no legacy BIOS mode). The iMac's firmware recognizes the Huey-Core's ESP as a bootable volume. Should the system ever need to boot macOS or another OS, one would use the Mac's boot picker, but as configured, Huey-Core's GRUB is first in the boot order. The HueyOS team has effectively transformed the iMac into a dedicated Linux machine with a custom boot workflow, shedding all Apple software.

## Known Limitations and Ongoing Tasks

While Huey-Core is fully operational, a few **limitations and open tasks** remain as the system is refined:

- **Audio Quirks (Mitigations in Place):** As discussed, the CS8409 audio requires custom initialization. The current solution works (speakers and headphone output function), but it's acknowledged as a **quirky workaround**. Automation of headphone sensing, internal mic support, and general elimination of `hda-verb` hacks is a goal. The project notes explicitly list iMac 5K audio quirks under 6.17.x as a known issue, with mitigation scripts in use and a long-term fix tracked in kernel tasks <sup>14</sup>. This likely means they are monitoring kernel mailing lists or even contributing patches to get proper support in future kernels. Until then, any major OS update might necessitate re-testing the audio setup. (**Open task:** integrate any upstream patch once available, remove the userland tweak script accordingly.)
- **Kernel Update & Debian Forky Migration:** Huey-Core is scheduled to move from Debian 13 to **Debian 14 "Forky"** and to solidify on kernel 6.17.x (or newer) as the baseline <sup>44</sup>. The target date (Oct 31, 2025) for this changeover indicates that at the time of writing, the migration is imminent or in progress. Upgrading the distribution can introduce changes (e.g., PipeWire might update, new drivers, etc.). An open task is to **validate the entire Huey-Core stack on Debian 14**. This includes verifying that the custom kernel 6.17.5 works with Forky's userspace or whether a rebuild to 6.18 or 6.19 (if Forky provides it) is needed. Another sub-task is updating the Python environment to 3.14 and ensuring all HueyOS Python dependencies are compatible <sup>45</sup>. Essentially, Huey-Core will be the testbed for the project's upgrade process.
- **Performance Tuning for GPU AI Workloads:** The iMac's AMD GPU (likely a Polaris 10 or 11 chip, similar to Radeon RX 570 series) is not a high-end compute GPU, but the project does plan to use Vulkan and ROCm for AI model inference where possible <sup>46</sup>. There is an ongoing effort to confirm that a quantized **Mistral-7B** model can run on this GPU via Vulkan backends <sup>47</sup>. If the performance is lacking or drivers pose issues (ROCm on Polaris might be limited), a limitation is that Huey-Core cannot run heavier AI tasks locally – those would be offloaded to Huey-Prime (with a beefier GPU) or run on CPU (slower). This is more of a performance note than a functional bug. (**Open task:** optimize or find workable ML frameworks for AMD GPU or consider an eGPU/Thunderbolt in future for more power.)
- **RAM Mode Sync & Update Process:** Using a SquashFS image means that whenever the base system is changed significantly (e.g., major Debian update or large package install), the SquashFS should be regenerated to keep things efficient. Currently, building a new SquashFS (and updating the GRUB config if needed) is likely a manual admin task. An improvement would be to automate this – e.g., a script that runs at shutdown or on demand to remaster the image from the current system state. Without it, over time the persistence partition accumulates a lot of changes and the base image becomes outdated. This doesn't break anything (the overlay will override old files with new ones), but it can slow boot (lots of data to overlay) and waste RAM (if toram, you're copying some unused old data). (**Open task:** implement a "remaster script" and document its use, possibly integrate into the update workflow.)
- **Disk Space Management:** The 16 GB system partition is tight but adequate. However, the 100 GB persistence could eventually fill up with logs, downloads, etc. Monitoring tools or alerts might be

needed. The project's memory management includes log rotation policies and provenance tagging <sup>48</sup>. Ensuring those are applied (so that, for example, old JSON log archives are compressed or moved off-system after a while) will prevent the persistence volume from running out of space. This is standard maintenance but worth noting as an ongoing operational task.

- **Live USB (Huey-Key) Upkeep:** The Huey key (live USB) should mirror the internal system's state to be truly useful as a fallback. If the internal system gets updated (new kernel, etc.), the USB should be recreated or updated. Automating the ISO build and writing it to USB is another task. The repository shows artifacts like `/.disk/info` and checksums for an ISO snapshot <sup>49</sup>, meaning they have a build process for it. The open item is to **polish the ISO builder and sign the artifacts** (ensuring integrity). Additionally, testing that the live USB can boot on various hardware (not just the iMac) is part of making HueyOS portable.
- **Miscellaneous Polish:** A few minor tasks remain, like adding a `.desktop` launcher for `huey-run` (so that if the system is restarted manually, a user can just double-click an icon to bring up the Huey runtime) <sup>50</sup>. Also, finalizing documentation for VNC workflow and other usage instructions is on the agenda <sup>51</sup>. These don't affect the core functionality but improve usability for anyone new coming to the project or for future maintainers.
- **Potential Hardware Limits:** The 2017 iMac lacks certain modern hardware like PCIe Gen4 or USB-C 3.2 speeds, etc. For the project's needs this is fine, but if new peripherals are added (e.g., high-bandwidth cameras or LiDAR), the USB 3.0 and Thunderbolt 3 ports are the conduits. Ensuring drivers for those (like Thunderbolt support in Linux, and any quirk for the iMac's Thunderbolt controller) is another check to consider. So far, no issues have been reported, but it's a watchpoint if Huey-Core ever needs to interface with hardware beyond its built-ins.

In conclusion, Huey-Core (iMac 5K) has been successfully transformed into a powerful Linux node anchoring the Monkey-Head Project's infrastructure. It combines a custom kernel, specialized boot setup, and a mixture of modern and retro software elements to meet the project's goals. The remaining issues – chiefly the fine-tuning of audio and the streamlining of update processes – are well in hand, with clear plans outlined to address them. As the project evolves, Huey-Core will continue to be the central hub, demonstrating how even aging proprietary hardware can be given a second life as an open-source robotics controller with carefully crafted system integration. The knowledge gained from this node will also feed into setting up other nodes and refining the HueyOS deployment process for future hardware. <sup>16</sup> <sup>17</sup>

---

<sup>1</sup> <sup>2</sup> <sup>9</sup> <sup>10</sup> <sup>14</sup> <sup>15</sup> <sup>16</sup> <sup>17</sup> <sup>19</sup> <sup>22</sup> <sup>27</sup> <sup>30</sup> <sup>32</sup> <sup>34</sup> <sup>35</sup> <sup>37</sup> <sup>38</sup> <sup>41</sup> <sup>44</sup> <sup>45</sup> <sup>46</sup> <sup>47</sup> <sup>48</sup> <sup>50</sup> <sup>51</sup> [GitHub](https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/README.md)  
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/README.md>

<sup>3</sup> <sup>4</sup> <sup>5</sup> <sup>6</sup> <sup>7</sup> <sup>8</sup> <sup>11</sup> <sup>12</sup> <sup>13</sup> [GitHub](https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/docs/kernel-6.17.3-runbook.md)  
[https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/docs/](https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/docs/kernel-6.17.3-runbook.md)  
[kernel-6.17.3-runbook.md](https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/docs/kernel-6.17.3-runbook.md)

<sup>18</sup> <sup>20</sup> <sup>21</sup> <sup>39</sup> <sup>40</sup> <sup>42</sup> <sup>43</sup> [GitHub](https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/huey/memory/CFG/grub.cfg)  
[https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/huey/memory/](https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/huey/memory/CFG/grub.cfg)  
[CFG/grub.cfg](https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/huey/memory/CFG/grub.cfg)

23 24 25 26 28 29 **GitHub**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/docs/api-reference.md>

31 **GitHub**

[https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/docs/network\\_setup.md](https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/docs/network_setup.md)

33 **GitHub**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/docs/kernel-upgrade-phase2.md>

36 **GitHub**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/huey/memory/MD/os-debloating.md>

49 **GitHub**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/2a1b1f64bbf63112f7ec18e9a3b30709cc6e974b/.disk/info>